CSA02 – C PROGRAMMING ASSIGNMENT 2 REPORT

Student Name: Sri Varshini K.
Register Number: 192512328
Course: C Programming – CSA0223
Compiler/IDE: Dev-C++ on Windows
GitHub Repository: CSA02-192512328-Assignment2
Repository URL: https://github.com/varshinikdas/CSA02-192512328-Assignment2.git

---

INTRODUCTION

This report presents the analysis, design, implementation, testing and evaluation of seven C programming tasks. The programs demonstrate arithmetic operator precedence, short-circuit evaluation, bitwise operations, decision-making statements, loop optimisation, loan repayment calculations, and algorithm redesign.

Each task includes its objective, problem analysis, design or pseudocode, source code, test cases, expected output, alternative concepts, complexity analysis and a space for inserting the program output screenshot.

The programs were developed for execution using Dev-C++ on Windows.

QUESTION 1 – PHARMACY BILLING AND OPERATOR PRECEDENCE

1.1 Problem Analysis

The task is to develop a pharmacy billing program that calculates the bill amount for three customers. The program accepts the price per item, quantity purchased and senior-citizen status.

The calculation applies a 15% senior-citizen reduction when the senior flag is 1 and adds 5% of the gross amount. If the calculated total exceeds Rs.1000, a further reduction of Rs.50 is applied to determine the net payable amount.

The program demonstrates arithmetic operators, operator precedence, associativity, the conditional operator and repeated input processing.

1.2 Objectives

- Accept customer billing details.
- Calculate the total using arithmetic expressions.
- Demonstrate operator precedence and associativity.
- Apply a conditional reduction when the total exceeds Rs.1000.
- Display the total and net payable amount.

1.3 Algorithm / Pseudocode

1. Start.

2. Repeat the following steps for three bills.
3. Read the item price, quantity and senior-citizen flag.
4. Calculate the total using the specified arithmetic expression.
5. If the total is greater than Rs.1000, subtract Rs.50.
6. Otherwise, keep the total unchanged as the net payable amount.
7. Display the total and net payable amount.
8. Stop.

1.4 Source Code – Q1.c

```
#include <stdio.h>

int main(void)
{
    int price, qty, senior;
    int total, net;
    int i;

    printf("JAN AUSHADHI PHARMACY BILLING\n");

    for (i = 1; i <= 3; i++)
    {
        printf("\nEnter details for Bill %d\n", i);

        printf("Enter price per item: ");
        scanf("%d", &price);

        printf("Enter quantity: ");
        scanf("%d", &qty);

        printf("Senior citizen? (1 = Yes, 0 = No): ");
        scanf("%d", &senior);

        total = price * qty
              - price * qty * 15 / 100 * senior
              + price * qty * 5 / 100;

        net = total > 1000 ? total - 50 : total;

        printf("\nTotal amount = Rs.%d\n", total);
        printf("Net payable = Rs.%d\n", net);
    }

    return 0;
}
```

1.5 Operator Precedence and Associativity

Operator precedence determines which operators are grouped first in an expression. Multiplication and division have higher precedence than addition and subtraction.

For the expression:

```
price * qty - price * qty * 15 / 100 * senior + price * qty * 5 / 100
```

The multiplication and division operations are grouped before the addition and subtraction operations. Operators of equal precedence in these arithmetic groups associate from left to right.

The conditional expression:

```
net = total > 1000 ? total - 50 : total;
```

assigns `total - 50` to `net` if the condition is true; otherwise, it assigns `total`.

Important observation: Integer arithmetic discards fractional parts. The program implements the expression exactly as written. If the intended billing policy requires applying tax to an already discounted subtotal, the calculation should instead be performed in separate, clearly defined stages.

1.6 Test Cases and Expected Results

Price	Quantity	Senior flag	Expected total	Expected net
100	2	1	180	180
250	4	1	900	900
300	2	0	630	630
600	2	0	1260	1210

The first three cases produce totals below Rs.1000, so the additional Rs.50 reduction is not applied. In the fourth case, the total is Rs.1260 and the net payable amount becomes Rs.1210.

1.7 Alternative Concept and Comparative Analysis

An alternative design is to calculate the gross amount, senior-citizen reduction, tax and final payable amount in separate variables.

Aspect	Single arithmetic expression	Separate calculation stages
Code length	Shorter	Slightly longer
Readability	Can be difficult for beginners	Easier to understand
Debugging	Individual components are harder to inspect	Each component can be checked
Policy changes	Expression may need rewriting	Individual stages are easier to modify

The single expression is compact, while separate stages are generally more maintainable for a real billing system.

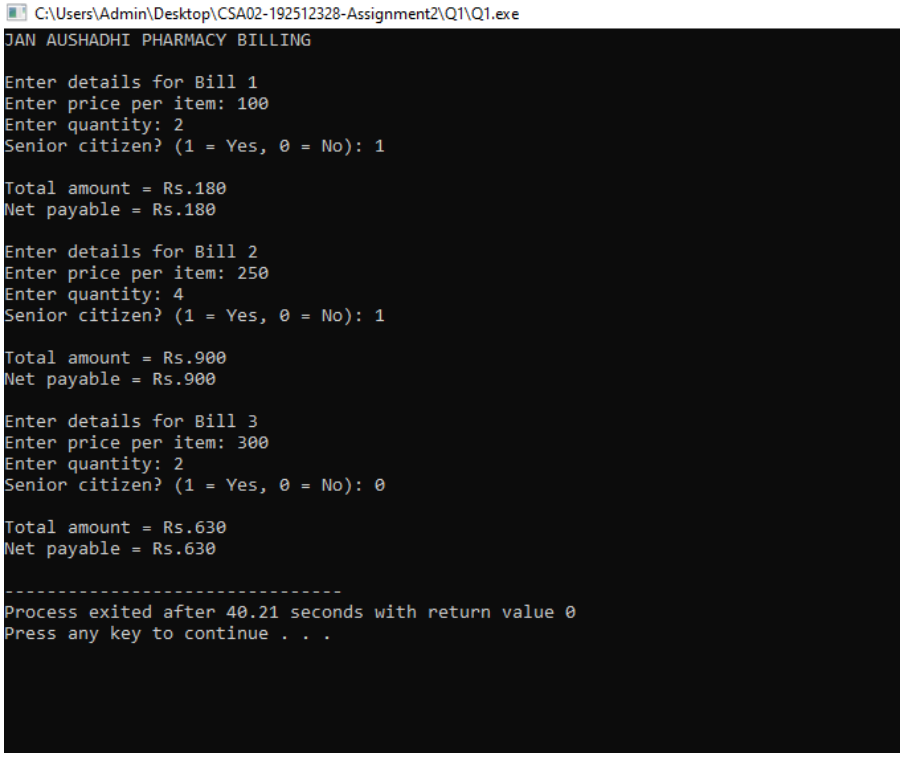
1.8 Time and Space Complexity

For a fixed three bills, the program takes constant time and uses constant auxiliary space.

- **Time complexity:** O(1) for the fixed three-bill program.
- **Space complexity:** O(1).

For a generalised version processing B bills, the time complexity is O(B), while auxiliary space remains O(1).

1.9 Actual Result and Screenshot



QUESTION 2 – SHORT-CIRCUIT EVALUATION AND FUNCTION-CALL COUNTING

2.1 Problem Analysis

The task demonstrates how logical AND (`&&`) and logical OR (`||`) use short-circuit evaluation. In C, the right operand of `&&` is not evaluated if the left operand is false. Similarly, the right operand of `||` is not evaluated if the left operand is true.

A verification function increments a counter whenever it is called. This makes it possible to measure how often a verification operation occurs and compare different condition orders.

2.2 Objectives

- Demonstrate short-circuit evaluation.
- Count calls to a verification function.
- Compare different condition orders.
- Compare logical AND (`&&`) with bitwise AND (`&`).
- Explain the relationship between execution efficiency and verification requirements.

2.3 Algorithm / Pseudocode

1. Start.
2. Define a verification function that increments a counter.
3. Evaluate `0 && function()` and display the result and call count.
4. Evaluate `1 || function()` and display the result and call count.
5. Simulate 1000 passengers using senior-citizen and eligibility conditions.
6. Count verification calls when eligibility conditions are checked first.

7. Repeat with the card-verification function checked first.
8. Compare logical AND with bitwise AND.
9. Display the results and stop.

2.4 Source Code – Q2.c

```
#include <stdio.h>

int calls;

int verifyDisabilityCard(int card)
{
    calls++;
    return card;
}

int main(void)
{
    int i;
    int senior, eligible, card;
    int result;

    int seniorData[5]   = {0, 1, 1, 0, 1};
    int eligibleData[5] = {1, 1, 0, 1, 1};
    int cardData[5]     = {0, 0, 1, 1, 1};

    printf("RAILWAY SENIOR-CITIZEN CONCESSION\n");

    printf("\nExperiment 1: Short-circuit AND (&&)\n");
    calls = 0;
    result = 0 && verifyDisabilityCard(1);
    printf("Result = %d, Function calls = %d\n", result, calls);

    printf("\nExperiment 2: Short-circuit OR (||)\n");
    calls = 0;
    result = 1 || verifyDisabilityCard(1);
    printf("Result = %d, Function calls = %d\n", result, calls);

    printf("\nExperiment 3: Condition order for 1000 passengers\n");

    calls = 0;
    for (i = 0; i < 1000; i++)
    {
        senior = (i % 4 != 0);
        eligible = (i % 3 != 0);
        card = (i % 10 == 0);

        if ((senior && eligible) ||
            verifyDisabilityCard(card))
        {
            /* Concession granted */
        }
    }
    printf("Senior conditions first: %d verification calls\n", calls);

    calls = 0;
    for (i = 0; i < 1000; i++)
    {
        senior = (i % 4 != 0);
        eligible = (i % 3 != 0);
        card = (i % 10 == 0);

        if (verifyDisabilityCard(card) ||
            (senior && eligible))
        {
            /* Concession granted */
```

```
    }
}
printf("Card verification first: %d verification calls\n", calls);

printf("\nExperiment 4: Bitwise & versus logical &&\n");

calls = 0;
result = 0 && verifyDisabilityCard(1);
printf("Using &&: result = %d, calls = %d\n", result, calls);

calls = 0;
result = 0 & verifyDisabilityCard(1);
printf("Using &: result = %d, calls = %d\n", result, calls);

return 0;}
```

2.5 Expected Results

Test	Expected result	Explanation		
`0 && function()`	Result 0, calls 0	The function is skipped		
`1		function()`	Result 1, calls 0	The function is skipped
Eligibility conditions first	500 calls	Verification occurs only when the preceding combined condition is false		
Card verification first	1000 calls	Verification occurs for every simulated passenger		
`0 && function()`	Calls 0	Logical AND short-circuits		
`0 & function()`	Calls 1	Bitwise AND evaluates both operands		

2.6 Comparative Analysis

Logical operators are useful for combining conditions while avoiding unnecessary evaluations. Bitwise operators operate on individual bits and do not provide logical short-circuit behaviour.

Changing condition order may reduce function calls. However, a lower call count is not always the correct objective. If every disability card must be verified for auditing or compliance, the verification must not be skipped.

Aspect	Logical short-circuit evaluation	Explicit verification
Function calls	May be reduced	Can ensure every required check occurs
Performance	Often more efficient	May require more work
Auditing	Depends on which checks are skipped	Better when every record must be verified
Suitable use	Optional or expensive conditions	Mandatory validation and auditing

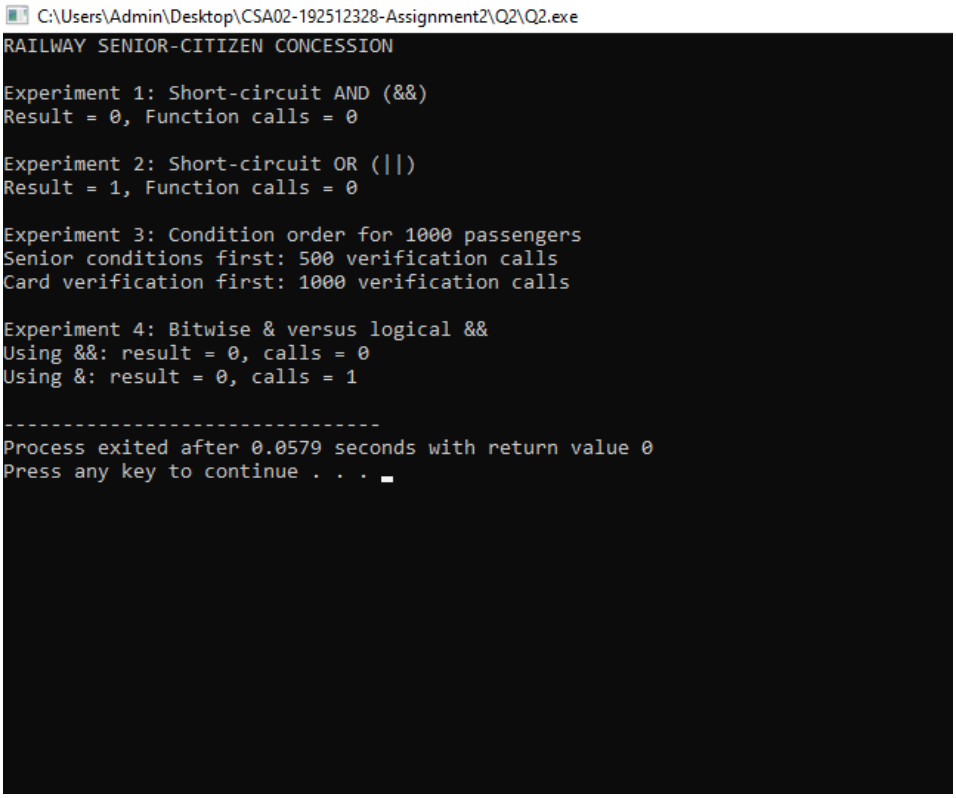
2.7 Time and Space Complexity

For N passengers:

- **Time complexity:** O(N).
- **Space complexity:** O(1).

The two simulations process a fixed number of operations for each passenger.

2.8 Actual Result and Screenshot



QUESTION 3 – BITWISE OPERATIONS AND REGISTER TRACING

3.1 Problem Analysis

The task models the ON/OFF state of electrical devices using a single unsigned byte. Each bit represents one device. Bitwise operations are used to turn devices on, turn them off, toggle their states and check whether they are on.

The program demonstrates bit masks, left shifts, bitwise OR, bitwise AND, bitwise XOR and binary register tracing.

3.2 Objectives

- Represent device states using bits.
- Use bitwise OR to turn a bit on.
- Use bitwise AND and complement to turn a bit off.
- Use bitwise XOR to toggle a bit.
- Display the register in binary and inspect device states.

3.3 Bit Mapping

Device	Bit position	Mask
LIGHT	0	00000001
FAN	1	00000010
AC	2	00000100
HEATER	3	00001000
PUMP	4	00010000

A bit value of 1 means ON, while a bit value of 0 means OFF.

3.4 Algorithm / Pseudocode

1. Start with the register set to zero.
2. Turn LIGHT on.
3. Turn AC on.
4. Display the register in binary.
5. Toggle FAN.
6. Turn LIGHT off.
7. Display the updated register.
8. Check and display the states of AC and PUMP.
9. Stop.

3.5 Source Code – Q3.c

```
#include <stdio.h>

/* Each bit stores the ON/OFF state of one device */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
    int bit;

    for (bit = 7; bit >= 0; bit--)
        putchar(((reg >> bit) & 1u) ? '1' : '0');

    putchar('\n');
}
```

```
int main(void)
{
    unsigned char reg = 0;

    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);

    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);

    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");

    return 0;
}
```

3.6 Register Trace and Expected Output

Operation	Binary register	Decimal value
Initial state	00000000	0
Turn LIGHT on	00000001	1
Turn AC on	00000101	5
Toggle FAN on	00000111	7
Turn LIGHT off	00000110	6

Expected output:

```
00000101
00000110
AC is ON, PUMP is OFF
```

3.7 Alternative Concept and Comparative Analysis

An alternative is to store each device state in an array of integers or Boolean values.

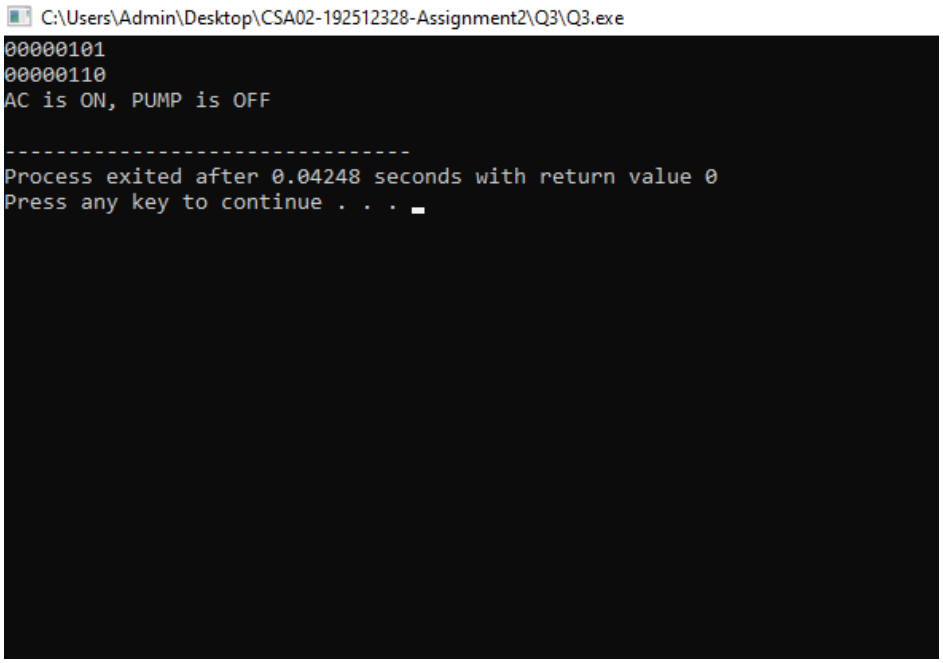
Aspect	Bitwise register	Array of device states
Memory	Compact representation	Usually uses more memory
Individual state changes	Requires masks and shifts	Easier to understand
Readability	Requires bitwise knowledge	More beginner-friendly
Scalability	Compact for many flags	Convenient for more complex state data

Bitwise operations are useful when memory efficiency matters or when working with hardware registers. Arrays are often easier to maintain when each device needs additional information.

3.8 Time and Space Complexity

- Turning on, turning off, toggling or checking a bit: O(1).
- Printing a fixed eight-bit register: O(1).
- Auxiliary space: O(1).

3.9 Actual Result and Screenshot



QUESTION 4 – UNIVERSITY GRADE CLASSIFICATION

4.1 Problem Analysis

The task classifies student marks into grades. It compares two approaches: a `switch` statement and an `else-if` ladder. Both approaches must produce the same grade for valid marks and identify invalid marks.

The grading ranges used are:

- 90–100: S
- 80–89: A
- 70–79: B
- 60–69: C
- 50–59: D
- 0–49: F
- Below 0 or above 100: Invalid (`?`)

4.2 Objectives

- Implement grade classification using `switch`.
- Implement grade classification using an `else-if` ladder.
- Test boundaries and invalid inputs.
- Compare readability and maintainability.

4.3 Algorithm / Pseudocode

1. Start.
2. Read or select a mark.
3. If the mark is below 0 or above 100, return `?`.

4. In the switch version, use integer division by 10 to select a grade group.
5. In the else-if version, compare the mark against the grade thresholds.
6. Display both grades.
7. Repeat for the test values.
8. Stop.

4.4 Source Code – Q4.c

```
#include <stdio.h>

char grade_switch(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';

    switch (marks / 10)
    {
        case 10:
        case 9:
            return 'S';

        case 8:
            return 'A';

        case 7:
            return 'B';

        case 6:
            return 'C';

        case 5:
            return 'D';

        default:
            return 'F';
    }
}

char grade_ladder(int marks)
{
    if (marks < 0 || marks > 100)
        return '?';
    else if (marks >= 90)
        return 'S';
    else if (marks >= 80)
        return 'A';
    else if (marks >= 70)
        return 'B';
    else if (marks >= 60)
        return 'C';
    else if (marks >= 50)
        return 'D';
    else
        return 'F';
}

int main(void)
{
    int marks[] = {100, 90, 89, 0, -1, 101, 95, 83, 50, 49, -5};
    int i;

    printf("UNIVERSITY GRADE CLASSIFIER\n\n");
    printf("Marks\tSwitch\tElse-if\n");

    for (i = 0; i < 11; i++)
    {
```

```
        printf("%d\t%c\t%c\n",
               marks[i],
               grade_switch(marks[i]),
               grade_ladder(marks[i]));
    }

    return 0;
}
```

4.5 Test Cases and Expected Results

Marks	Switch result	Else-if result	Purpose
100	S	S	Upper boundary
90	S	S	S-grade boundary
89	A	A	Below S boundary
83	A	A	Typical valid mark
70	B	B	B-grade boundary
60	C	C	C-grade boundary
50	D	D	D-grade boundary
49	F	F	Below D-grade boundary
0	F	F	Lower valid boundary
-1	?	?	Invalid lower value
101	?	?	Invalid upper value

Both methods should return the same grade for every listed input.

4.6 Comparative Analysis

The `switch` version groups marks by `m / 10`. It is compact when categories can be represented by discrete integer values. The `else-if` ladder directly expresses ranges and may be easier to modify when grading thresholds change.

Aspect	Switch	Else-if ladder
Best suited for	Discrete cases	Ranges and complex conditions
Readability	Compact for grouped cases	Directly expresses thresholds
Range handling	Usually requires mapping to discrete cases	Direct range comparisons
Maintenance	Easy for stable categories	Convenient for irregular thresholds

A lookup table is another alternative. It can provide direct access for integer marks from 0 to 100, but the input must be checked before indexing the table.

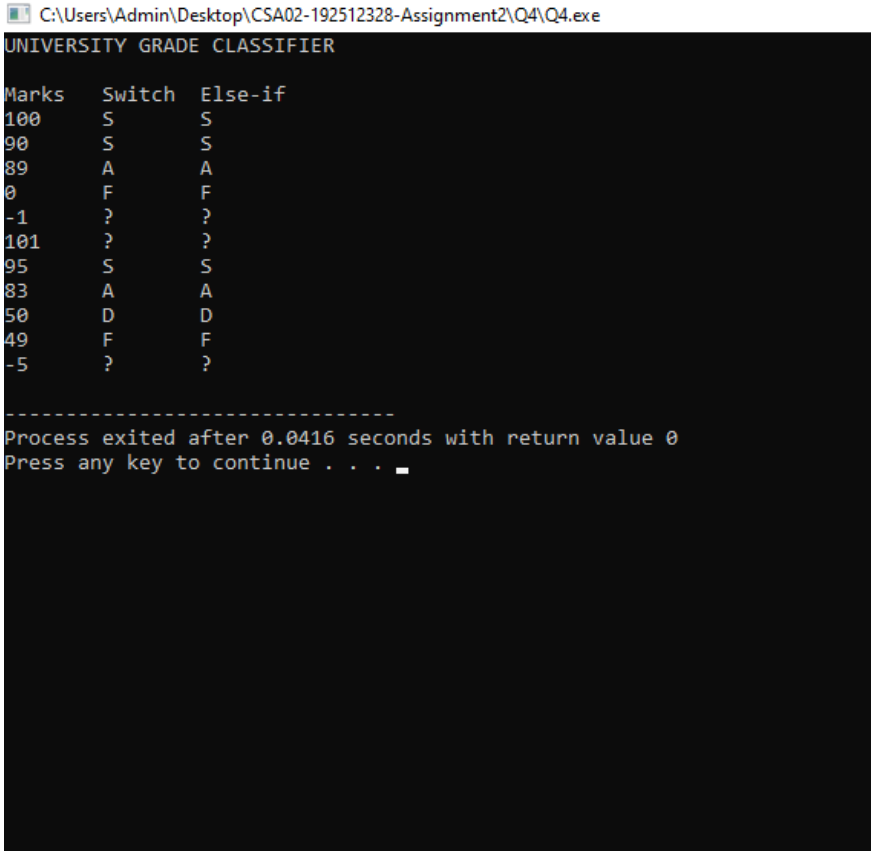
4.7 Time and Space Complexity

Because both methods use a fixed number of conditions:

- **Time complexity:** O(1).
- **Space complexity:** O(1), excluding the fixed test array.

For a generalised decision ladder containing k conditions, the worst-case time can be O(k).

4.8 Actual Result and Screenshot



QUESTION 5 – PRIME NUMBER VERIFICATION AND LOOP OPTIMISATION

5.1 Problem Analysis

The task checks whether a number is prime and compares a basic trial-division method with an improved method.

The basic method tests every possible divisor from 2 to n−1. The improved method handles small and even values separately and tests only odd divisors up to the square root of the number.

The number of divisibility checks is counted to measure the improvement.

5.2 Objectives

- Identify prime and non-prime numbers.
- Compare basic and improved trial division.
- Count divisibility checks.
- Measure the effect of reducing unnecessary loop iterations.
- Explain time and space complexity.

5.3 Algorithm / Pseudocode

Basic method

1. If n is less than 2, return not prime.
2. Test every integer divisor from 2 to n−1.
3. If any divisor divides n exactly, return not prime.
4. Otherwise, return prime.

Improved method

1. If n is less than 2, return not prime.
2. If n equals 2, return prime.
3. If n is even, return not prime.
4. Test odd divisors starting from 3 while the divisor is no greater than n divided by the divisor.
5. If a divisor divides n exactly, return not prime.
6. Otherwise, return prime.

5.4 Source Code – Q5.c

```
#include <stdio.h>

unsigned long long basicChecks, improvedChecks;

int basicPrime(unsigned long long n)
{
    unsigned long long i;

    basicChecks = 0;

    if (n < 2)
        return 0;

    for (i = 2; i < n; i++)
    {
        basicChecks++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int improvedPrime(unsigned long long n)
{
    unsigned long long i;

    improvedChecks = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0)
    {
        improvedChecks++;
        return 0;
    }

    for (i = 3; i <= n / i; i += 2)
    {
        improvedChecks++;

        if (n % i == 0)
```

```
            return 0;
    }

    return 1;
}

int main(void)
{
    unsigned long long values[] = {97, 100, 7919, 1000003};
    int i, a, b;

    printf("PRIME VERIFICATION COMPARISON\n");
    printf("Number\tBasic\tChecks\tImproved\tChecks\n");

    for (i = 0; i < 4; i++)
    {
        a = basicPrime(values[i]);
        b = improvedPrime(values[i]);

        printf("%llu\t%s\t%llu\t%s\t%llu\n",
               values[i],
               a ? "Prime" : "Not prime",
               basicChecks,
               b ? "Prime" : "Not prime",
               improvedChecks);
    }

    return 0;
}
```

5.5 Test Cases and Expected Results

Number	Expected classification	Basic checks	Improved checks
97	Prime	95	4
100	Not prime	1	1
7919	Prime	7917	43
1000003	Prime	1000001	499

For 97, the improved method tests 3, 5, 7 and 9 before concluding that no divisor exists. For 100, the improved method detects that the number is even after one check.

For the prime number 1,000,003, the basic method performs 1,000,001 checks, while the improved method performs 499. Both methods should return the same classification.

5.6 Comparative Analysis

Aspect	Basic trial division	Improved trial division
Divisors checked	From 2 to n−1	Odd divisors up to √n
Time complexity	O(n)	O(√n)
Auxiliary space	O(1)	O(1)
Main advantage	Simple implementation	Much faster for large individual numbers
Limitation	Performs many unnecessary checks	Still slower than specialised methods for some large-scale workloads

An alternative is the Sieve of Eratosthenes. It is useful when many prime numbers up to a fixed limit are required. It takes O(N log log N) time and O(N) space, so it uses more memory than checking one number with trial division.

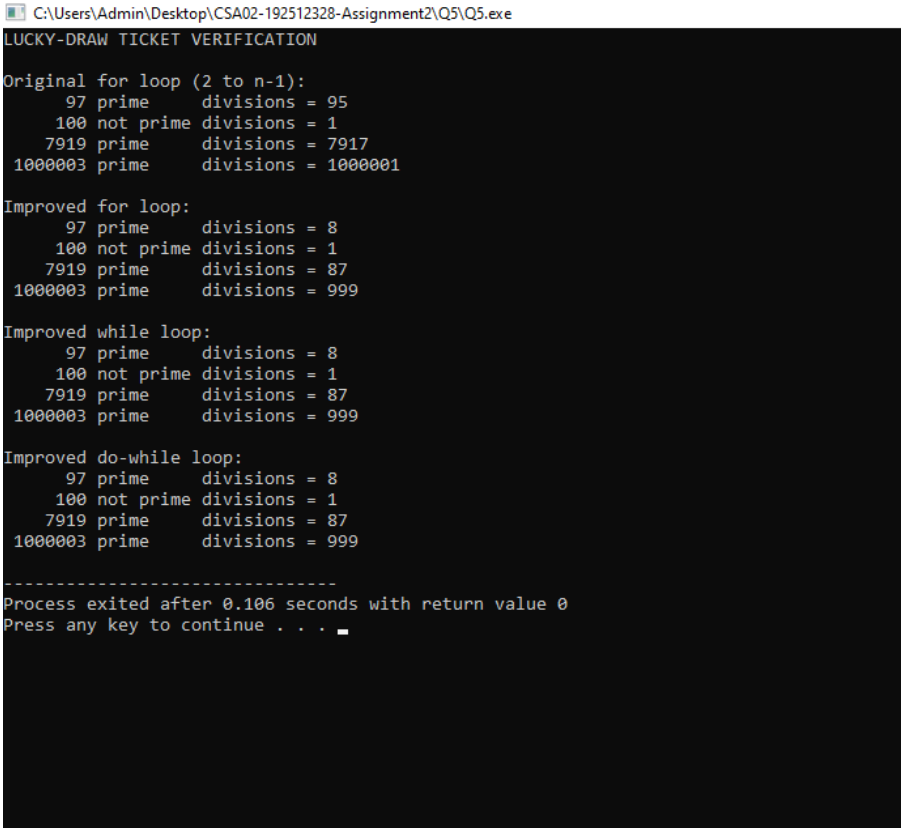
5.7 Experimental Analysis

The experiment measures the number of divisibility checks rather than relying only on execution time. This makes the comparison easier to reproduce because execution time can vary between computers and compiler configurations.

For the largest tested number:

- Basic checks: 1,000,001
- Improved checks: 499
- Checks avoided: 999,502
- Approximate reduction: 99.95%

5.8 Actual Result and Screenshot



QUESTION 6 – LOAN REPAYMENT SCHEDULE AND TERMINATION

6.1 Problem Analysis

The task calculates a loan repayment schedule using a principal amount, a monthly interest rate and a planned monthly repayment. Each month, interest is calculated on the opening balance and added to the balance. The repayment is then subtracted.

The program adjusts the final repayment so that it does not exceed the remaining amount. Input validation and a maximum iteration limit help prevent invalid calculations and an endless loop.

6.2 Objectives

- Accept the principal, interest rate and repayment amount.
- Validate input values.
- Calculate monthly interest.
- Display opening balance, interest, repayment and closing balance.
- Stop when the loan is cleared or the safety limit is reached.
- Calculate the total interest paid.

6.3 Algorithm / Pseudocode

1. Start.
2. Read the principal and validate that it is greater than zero.
3. Read the monthly interest rate and validate that it is not negative.
4. Read the planned monthly repayment and validate that it is greater than zero.
5. If the interest rate is positive and the payment cannot cover the first month's interest, reject the repayment.
6. Set the current balance equal to the principal.
7. While the balance remains above the rounding tolerance and the month limit has not been reached:
    - Calculate interest on the opening balance.
    - Add interest to the balance.
    - Set the actual payment to the planned payment or remaining balance, whichever is smaller.
    - Subtract the actual payment.
    - Update the total interest and month count.
    - Display the monthly schedule.
8. Report whether the loan has been cleared or the safety limit reached.
9. Display total interest and stop.

6.4 Source Code – Q6.c

```
#include <stdio.h>

int main(void)
{
    double principal, rate, payment, balance;
    double interest, actualPayment, totalInterest = 0.0;
    int month = 0;

    printf("LOAN REPAYMENT SCHEDULE\n");

    printf("Enter principal: ");
    if (scanf("%lf", &principal) != 1 || principal <= 0)
    {
        printf("Invalid principal.\n");
        return 1;
    }

    printf("Enter monthly interest rate in percent: ");
    if (scanf("%lf", &rate) != 1 || rate < 0)
    {
        printf("Invalid interest rate.\n");
        return 1;
    }
```

```
    printf("Enter planned monthly repayment: ");
    if (scanf("%lf", &payment) != 1 || payment <= 0)
    {
        printf("Invalid repayment.\n");
        return 1;
    }

    if (rate > 0 && payment <= principal * rate / 100.0)
    {
        printf("Repayment is too small to reduce the loan.\n");
        return 1;
    }

    balance = principal;

    printf("\nMonth\tOpening\tInterest\tPayment\tClosing\n");

    while (balance > 0.005 && month < 360)
    {
        double opening = balance;

        interest = opening * rate / 100.0;
        balance = opening + interest;

        actualPayment = payment;

        if (actualPayment > balance)
            actualPayment = balance;

        balance -= actualPayment;

        if (balance < 0.005)
            balance = 0.0;

        totalInterest += interest;
        month++;

        printf("%d\t%.2f\t%.2f\t%.2f\t%.2f\n",
               month, opening, interest,
               actualPayment, balance);
    }

    if (balance > 0.005)
    {
        printf("Stopped at safety limit; balance remains %.2f\n",
               balance);
    }
    else
    {
        printf("\nLoan cleared in %d months.\n", month);
    }

    printf("Total interest = %.2f\n", totalInterest);

    return 0;
}
```

6.5 Test Cases and Expected Results

Test	Input	Expected behaviour
Valid loan	Principal 10000, rate 2%, payment 2000	Monthly schedule is printed and loan is cleared
Invalid principal	Principal 0	Invalid principal message
Invalid rate	Rate −1%	Invalid interest rate message

Invalid repayment	Repayment 0	Invalid repayment message
Repayment too small	Principal 10000, rate 20%, payment 100	Program rejects repayment because it does not cover the first month's interest
Safety limit	A case that cannot clear within 360 months	Program reports the remaining balance

For the valid example, the first month's opening balance is Rs.10,000, interest is Rs.200, repayment is Rs.2,000 and closing balance is Rs.8,200.

A representative schedule is:

Month	Opening balance	Interest	Payment	Closing balance
1	10000.00	200.00	2000.00	8200.00
2	8200.00	164.00	2000.00	6364.00
3	6364.00	127.28	2000.00	4491.28
4	4491.28	89.83	2000.00	2581.11
5	2581.11	51.62	2000.00	632.73
6	632.73	12.65	645.38	0.00

The loan is cleared in six months, and the total interest is approximately Rs.645.38. Small display differences can occur due to floating-point arithmetic and rounding.

6.6 Alternative Concept and Comparative Analysis

An alternative is to use a mathematical amortisation formula to estimate the number of payments. A formula is useful when the repayment amount and interest rate remain fixed, but it does not automatically provide the same transparent month-by-month explanation as a loop.

Aspect	Monthly loop	Mathematical estimate
Monthly schedule	Directly generated	Requires additional calculation to generate
Transparency	Each balance update is visible	More dependent on formula interpretation
Complexity	O(M) for M months	O(1) for a direct duration estimate
Flexibility	Convenient for varying final payments	Depends on the assumptions of the formula

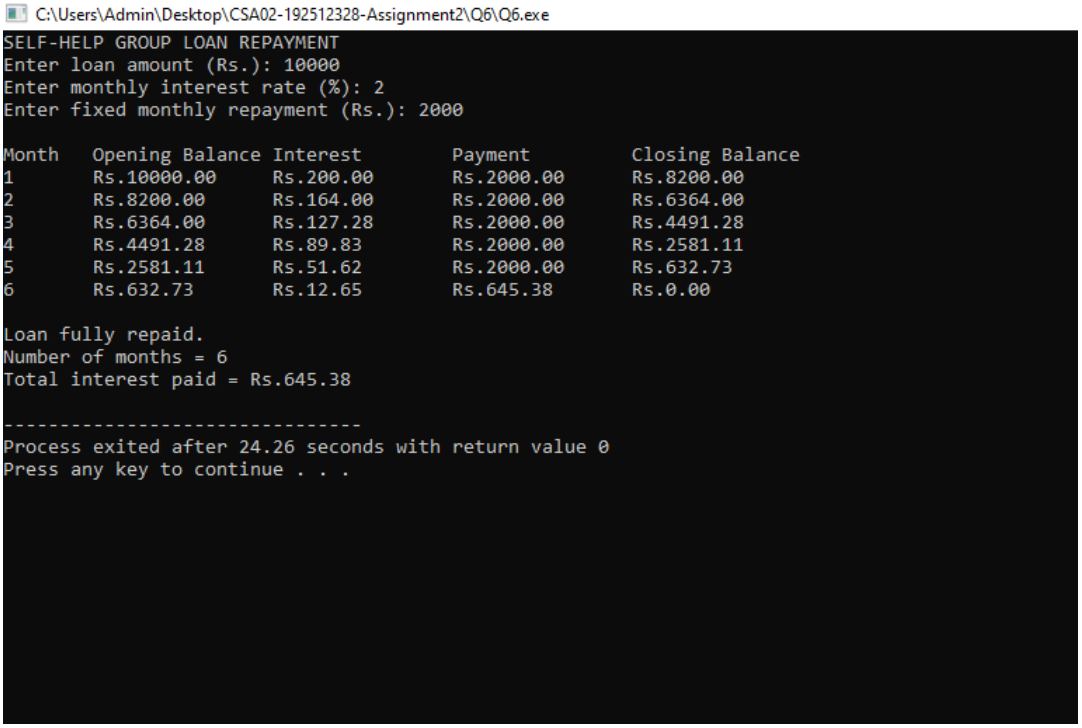
For financial applications, integer paise or an appropriate decimal-money representation is preferable to relying entirely on binary floating-point values.

6.7 Time and Space Complexity

- **Time complexity:** O(M), where M is the number of simulated months.
- **Space complexity:** O(1).

The loop stores only a fixed number of variables regardless of the number of months.

6.8 Actual Result and Screenshot



QUESTION 7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE

7.1 A. Diagnose the Original Design

The original prime-number verification design tests every integer divisor from 2 to n−1. Although the method produces the correct result for the tested values, it performs many unnecessary divisibility checks.

A composite number must have at least one factor that is no greater than its square root. Therefore, testing every divisor up to n−1 is inefficient, particularly for large prime numbers.

The primary limitation is the high number of repeated operations.

7.2 B. Define a Measurable Objective

The objective is to redesign the prime-checking method so that it produces the same prime or non-prime classification while reducing the number of divisibility checks.

The redesigned program will be tested using the values 97, 100, 7919 and 1,000,003.

The number of divisibility checks will be used as the main performance measurement.

7.3 C. Revised Design and Pseudocode

1. If n is less than 2, classify it as not prime.
2. If n equals 2, classify it as prime.
3. If n is even, count one parity check and classify it as not prime.
4. Start testing odd divisors at 3.
5. Continue while the divisor is no greater than n divided by the divisor.
6. If a divisor divides n exactly, classify the number as not prime.

7. If no divisor is found, classify the number as prime.

The condition `i <= n / i` is used instead of `i * i <= n` to avoid a possible multiplication overflow for large unsigned integers.

7.4 D. Predicted Improvement

The original method has O(n) time complexity. The redesigned method has O(√n) time complexity and O(1) auxiliary space.

It is predicted that the redesigned program will perform substantially fewer checks for large prime numbers while preserving the original classification.

7.5 E. Implementation and Validation

The following program compares the original and redesigned methods and counts the number of divisibility checks.

Source Code – Q7.c

```
#include <stdio.h>

unsigned long long originalChecks, improvedChecks;

int originalPrime(unsigned long long n)
{
    unsigned long long i;

    originalChecks = 0;

    if (n < 2)
        return 0;

    for (i = 2; i < n; i++)
    {
        originalChecks++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int redesignedPrime(unsigned long long n)
{
    unsigned long long i;

    improvedChecks = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0)
    {
        improvedChecks++;
        return 0;
    }
```

```
    for (i = 3; i <= n / i; i += 2)
    {
        improvedChecks++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int main(void)
{
    unsigned long long values[] = {97, 100, 7919, 1000003};
    int i, oldResult, newResult;

    printf("Q7: PRIME CHECK REDESIGN\n");
    printf("Number\tOriginal\tChecks\tRedesigned\tChecks\n");

    for (i = 0; i < 4; i++)
    {
        oldResult = originalPrime(values[i]);
        newResult = redesignedPrime(values[i]);

        printf("%llu\t%s\t%llu\t%s\t%llu\n",
               values[i],
               oldResult ? "Prime" : "Not prime",
               originalChecks,
               newResult ? "Prime" : "Not prime",
               improvedChecks);
    }

    return 0;
}
```

7.6 Validation Results

Input	Expected classification	Original checks	Redesigned checks	Checks avoided
97	Prime	95	4	91
100	Not prime	1	1	0
7919	Prime	7917	43	7874
1000003	Prime	1000001	499	999502

The original and redesigned methods return the same classification for every test input.

For 1,000,003, the original method performs 1,000,001 checks, whereas the redesigned method performs 499 checks. This is a reduction of approximately 99.95%.

The redesign is therefore supported by measurable evidence rather than only by a theoretical claim.

7.7 F. Defend the Final Design

The redesigned method is preferable for checking individual large numbers because it reduces the number of divisibility checks while retaining constant auxiliary space.

The design is justified by the following points:

- It preserves the classification results of the original method.
- It handles small and even values separately.
- It tests only odd divisors after handling even numbers.

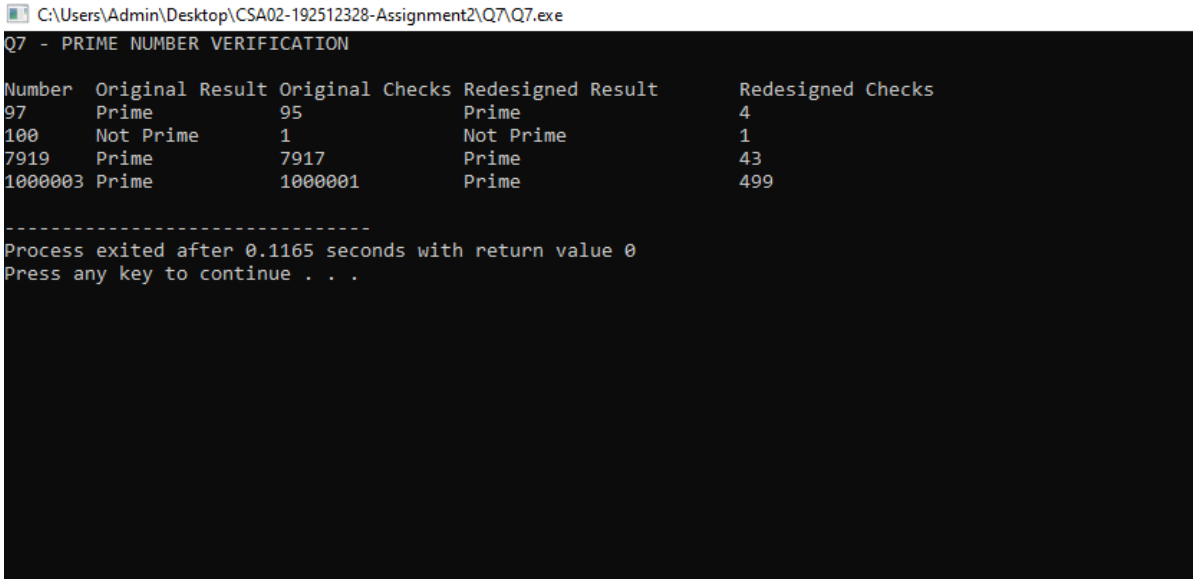
- It stops testing once the divisor exceeds the square-root boundary.
- It avoids possible overflow in the square-root condition by using `i <= n / i`.
- It reduces the time complexity from O(n) to O(√n).

For workloads requiring many prime queries up to a known maximum, a Sieve of Eratosthenes may be more suitable. However, it requires O(N) memory.

7.8 Comparative Analysis

Feature	Original design	Redesigned design
Divisor range	2 to n−1	Odd divisors up to √n
Time complexity	O(n)	O(√n)
Auxiliary space	O(1)	O(1)
Large prime performance	Many checks	Substantially fewer checks
Correctness for tested inputs	Correct	Same classifications
Main trade-off	Simple but inefficient	Slightly more logic but more efficient

7.9 Actual Result and Screenshot



OVERALL CONCLUSION

The seven tasks demonstrate important C programming concepts and the effect of design choices on program correctness, efficiency and maintainability.

Q1 demonstrated arithmetic operator precedence, associativity and conditional billing logic.
Q2 demonstrated short-circuit evaluation and the effect of condition ordering on function calls.
Q3 used bitwise operations to represent and manipulate device states efficiently.
Q4 compared `switch` and `else-if` structures for grade classification.
Q5 compared basic and improved prime-number verification.
Q6 demonstrated monthly loan calculations, input validation and controlled loop termination.
Q7 used diagnosis, redesign and measured validation to demonstrate the benefits of algorithmic improvement.
The comparative analyses show that the best solution depends not only on execution speed, but also on readability, memory requirements, correctness, maintainability and the purpose of the application.